
AWS IAM (Identity and Access Management) – Detailed Notes

What is AWS IAM?

AWS IAM (Identity and Access Management) is a **secure and scalable** service that enables you to control **who can access your AWS resources, what actions they can perform, and under what conditions.**

 IAM is **global**—it doesn't require region-specific setup.

1. IAM Users

What is an IAM User?

- Represents a **person or application**.
- Each user has:
 - A **name**
 - **Credentials** (password for AWS Console, access keys for CLI/API)
 - Optionally, **permissions** via policies

Use Cases

- Developers, Admins, Apps that need programmatic access
- Assign fine-grained access (e.g., S3 read-only, EC2 launch, etc.)

Best Practices

- One user = one human
 - Never use **root user** for daily tasks
 - Use **groups + policies**, not inline user permissions
-

2. IAM Groups

What is an IAM Group?

- A collection of IAM users
- Policies attached to a group apply to **all members**
- Simplifies management

Example:

Group Name	Attached Policy	Members
Admins	AdministratorAccess	Alice, Bob
DevTeam	AmazonS3ReadOnlyAccess	Charlie, Diana

Best Practices

- Use groups to assign roles like **Admin**, **ReadOnly**, **DevOps**, etc.
 - Update policy in one place → All members affected ✓
-

3. IAM Policies

What is a Policy?

- A **JSON document** that defines **permissions**
- Specifies:
 - **Actions:** What can be done (e.g., `s3:PutObject`)
 - **Resources:** Where (e.g., `arn:aws:s3:::mybucket/*`)
 - **Effect:** Allow or Deny
 - **Conditions:** Optional constraints (e.g., IP, time, MFA)

Types of Policies:

1. Managed Policies

- **AWS-Managed** (e.g., `AmazonS3ReadOnlyAccess`)
- **Customer-Managed** (your own reusable policies)

2. Inline Policies

- Attached to a single user/group/role
- Not reusable

Best Practices

- Prefer **managed policies** for consistency
- Use **least privilege principle** (grant only what's needed)
- Use **resource-level permissions** (e.g., only 1 S3 bucket)

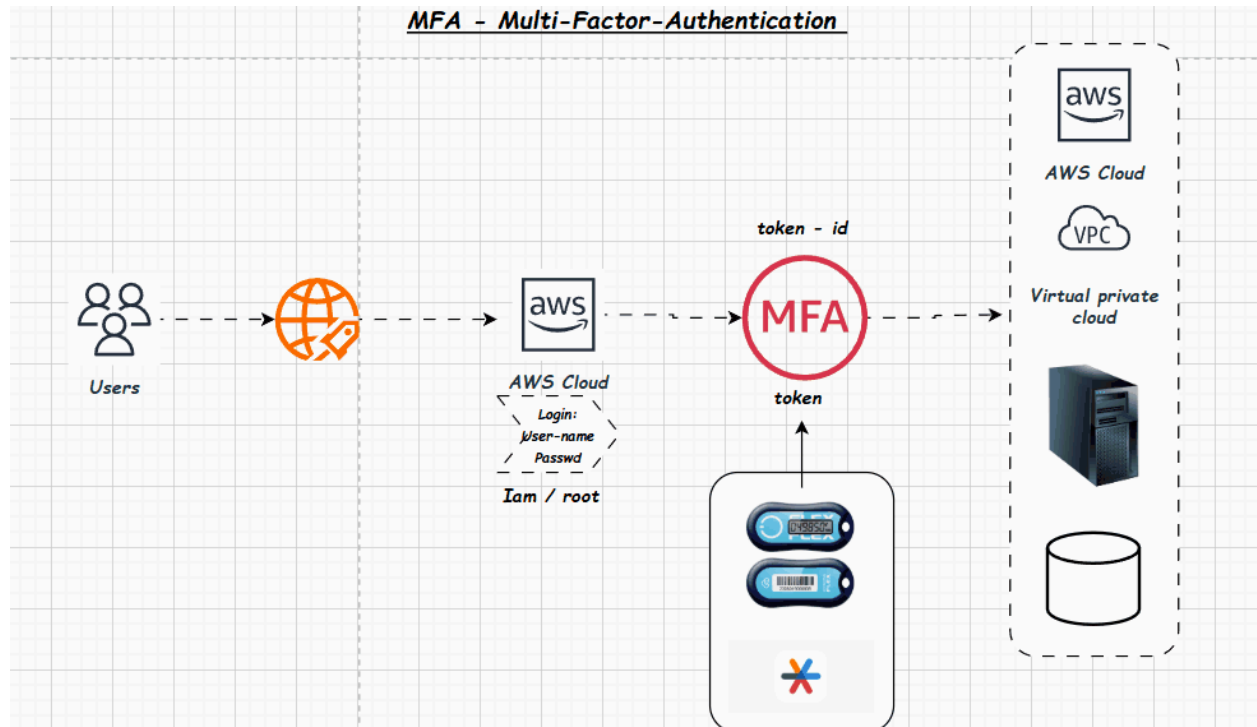
Policy Example

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "s3:GetObject",
        "s3:PutObject"
      ],
      "Resource": "arn:aws:s3:::my-bucket/*"
    }
  ]
}
```

 Meaning: Allows user to **read and write objects** to **my-bucket**

4. Multi-Factor Authentication (MFA)

 What is MFA?



- An extra layer of **security** using:
 - Something you **know** (password)
 - Something you **have** (MFA device)

📱 Supported MFA Devices:

- **Virtual MFA apps** (e.g., Google Authenticator, Authy)
- **Hardware tokens**
- **SMS-based MFA** (only for root user)

📄 Enabling MFA:

1. Go to **IAM Console** → **Users**

2. Select user → **Security credentials**
3. Click **Manage MFA device**
4. Choose **virtual or hardware device**
5. Scan QR code in authenticator app
6. Enter 2 consecutive OTPs → Done 

Best Practices

- **Enable MFA for all IAM users**
- **Mandatory for root user**
- Use **policy conditions** to **enforce MFA**

Example Condition:

```
"Condition": {  
  "Bool": {  
    "aws:MultiFactorAuthPresent": "true"  
  }  
}
```

IAM Key Concepts Summary

Concept	Description
User	Entity representing a person/app
Group	Collection of users for permission management
Policy	JSON doc defining access rules

MFA Second authentication factor for extra security

IAM Best Practices (for Production)

- ✓ Use **Groups** to assign permissions
 - ✓ Apply **Least Privilege** principle
 - ✓ Always enable **MFA**, especially for **root**
 - ✓ Monitor access using **CloudTrail**
 - ✓ Rotate **access keys** regularly
 - ✓ Avoid **inline policies** unless necessary
 - ✓ Use **IAM Roles** for EC2, Lambda, etc., instead of hardcoding keys
-

Want a Hands-On Lab?

Here's a quick **IAM Lab** idea:

- Create a user **dev-user**
- Add to a group **S3-ReadOnly**
- Attach **AmazonS3ReadOnlyAccess** to group
- Enable MFA for the user
- Log in as **dev-user** and test S3 access